
AlgRemez のコード読解

C++ 実装が解いている問題、アルゴリズム、
および Julia 移植時の論点

コード調査メモ

2026年9月15日

概要

本稿は、AlgRemez/src にある実装をコードに即して説明するものである。このプログラムは、正の区間上の $f(x)=x^{p/q}$ （および内部 API では指数因子を掛けた関数）に対し、指定した分子次数と分母次数をもつ有理関数を構成する。最小化対象は絶対誤差ではなく相対誤差の二乗ノルムである。実装上は、誤差の零点で補間問題を解き、その零点を反復的に移動して、隣接区間にある誤差極値の絶対値を揃える Remez 型の交換法になっている。収束後には実根を Newton 法で求め、次数が等しい場合に有理関数とその逆数の部分分数展開を出力する。なお、ファイルの拡張子と実装内容から分かるとおり、これは C ではなく C++ のコードである。

目次

調査対象と全体像

近似している数学的問題

対象関数

有理関数の表現

最小化する誤差

Remez 反復の実装

誤差零点での補間

初期点

各区間の誤差極値探索

零点の交換・更新

停止条件

擬似コード

連立一次方程式の解法

根と部分分数展開

根の計算

部分分数展開

コマンドラインプログラムの入出力

多倍長演算の実装

Julia 移植前に把握すべき制約と不具合

Julia 版へ移す際の推奨分割

コード位置との対応

要約

調査対象と全体像

調査対象は次のファイルである。

ファイル	役割
src/main.C	コマンドライン入力、計算実行、結果のファイル出力
src/alg_remez.h	AlgRemez クラスの宣言
src/alg_remez.C	Remez 反復、線形方程式、根、部分分数展開
src/bigfloat.h	GMP/MPFR を含む多倍長浮動小数点クラス
src/Makefile	GMP/MPFR とリンクする古い形式のビルド規則
README	利用方法、制限、引用情報

処理の流れは以下である。

1. 区間、次数、指数、有効桁数を読み込む。
2. 誤差零点と誤差極値位置の初期値を Chebyshev 点に似た配置で作る。
3. 現在の誤差零点で $R(x_i)=f(x_i)$ となる有理関数を線形方程式から求める。
4. 各零点間で相対誤差の極値を探索する。
5. 極値の大きさが等しくなる方向へ零点を移動し、収束まで 3-4 を繰り返す。
6. 分子・分母の実根を Newton 法と逐次デフレーションで求める。
7. $n=d$ の場合、有理関数とその逆数を部分分数展開する。
8. `approx.dat` と `error.dat` を出力する。

近似している数学的問題

対象関数

通常の公開インターフェースが対象とする関数は

$$f(x)=x^\alpha, \quad \alpha=\frac{p}{q}, \quad p,q\in\mathbb{Z}_{>0},$$

(1)

であり、近似区間は

$$x\in[a,b]$$

(2)

である。典型用途では $0 < a < b$ を仮定しているが、コード自身はこの条件を検査しない。負の指数 $x^{p/q}$ は直接生成せず、 $x^{p/q}$ の近似の逆数を使う。

引数の多い `generateApprox` には、さらに

$$f(x) = x^{p/q} \exp(\sum_{j=0}^{L-1} a_j x^{k_j}), \quad L \leq 10, \quad (3)$$

を扱う内部機能がある。ただし `main.C` は常に $L=0$ とするため、コマンドラインからこの機能は使われない。

有理関数の表現

分子次数 n 、分母次数 d の有理関数

$$R_{n,d}(x) = \frac{P_n(x)}{Q_d(x)} = \frac{\sum_{j=0}^n p_j x^j}{x^d + \sum_{j=0}^{d-1} q_j x^j} \quad (4)$$

を使う。分母の最高次係数を 1 に固定して全体のスケールの不定性を除いている。未知数は

$$(p_0, \dots, p_n, q_0, \dots, q_{d-1}) \quad (5)$$

の $K=n+d+1$ 個であり、コードでは `param` にこの順番で格納される。`approx` は分子・分母を Horner 法で評価する。

最小化する誤差

`getErr` が計算する符号付き相対誤差は

$$\varepsilon(x) = \frac{R_{n,d}(x) - f(x)}{f(x)} = \frac{R_{n,d}(x)}{f(x)} - 1 \quad (6)$$

である。探索ルーチンが実際に返す値は $E(x) = |\varepsilon(x)|$ である。したがって目標は

$$\min_{R_{n,d}} \max_{x \in [a,b]} |\varepsilon(x)| \quad (7)$$

という相対誤差の minimax 近似である。これは重み $w(x) = 1/f(x)$ をもつ重み付き一様近似と見なせる。

非退化な最良近似では、通常 $K+1=n+d+2$ 個の点で符号が交互に変わる等振幅誤差 (equioscillation) が期待される。このコードはその状態を、誤差の K 個の零点を動かすことで作ろうとしている。

Remez 反復の実装

誤差零点での補間

現在の誤差零点候補を

$$a < \xi_0 < \xi_1 < \dots < \xi_{K-1} < b \quad (8)$$

とする。 `equations` は各点で

$$R_{n,d}(\xi_i) = f(\xi_i), \quad i=0, \dots, K-1 \quad (9)$$

を課す。式 (4) の分母を払うと、各行は

$$\sum_{j=0}^n p_j \xi_i^j - f(\xi_i) \sum_{j=0}^{d-1} q_j \xi_i^j = f(\xi_i) \xi_i^d \quad (10)$$

となる。これは未知係数について線形であり、 K 本の連立一次方程式を構成する。

重要なのは、典型的な Remez の説明で用いられる誤差振幅 Δ を 未知数として直接解いてはいないことである。現在の零点で対象関数を正確に補間し、その後零点を動かすという実装である。

初期点

`initialGuess` は Chebyshev 点を基にしつつ、低い x 側へ点を寄せる変換

$$\varphi(t) = \frac{e^t - 1}{e - 1}, \quad 0 \leq t \leq 1 \quad (11)$$

を使う。初期の極値候補 m_i は

$$\begin{aligned} t_i^{(m)} &= \frac{1}{2} \left(1 - \cos \frac{\pi i}{K} \right), \quad i=0, \dots, K, \\ m_i &= a + (b-a) \varphi(t_i^{(m)}) \end{aligned} \quad (12)$$

である。初期の零点候補は

$$\begin{aligned} t_i^{(\xi)} &= \frac{1}{2} \left(1 - \cos \frac{\pi(2i+1)}{2K} \right), \\ \xi_i &= a + (b-a) \varphi(t_i^{(\xi)}), \quad i=0, \dots, K-1 \end{aligned} \quad (13)$$

である。実装は $i=K$ の値も配列に書くが、反復で使用する零点は $\xi_0, \dots, \xi_{K-1}$ だけである。

指数関数による変換そのものは C++ の `double` で計算されてから 多倍長型へ代入される。このため、初期点の精度は指定した多倍長精度ではなく、実質的に倍精度へ制限される。ただし初期値なので、収束すれば最終精度を直接制限するとは限らない。

各区間の誤差極値探索

零点によって区間を

$$[a, \xi_0], [\xi_0, \xi_1], \dots, [\xi_{K-2}, \xi_{K-1}], [\xi_{K-1}, b] \quad (14)$$

の $K+1$ 個に分ける。search は各区間で、前回の極値位置 m_i と ステップ s_i から出発し、 $E(x)=|\varepsilon(x)|$ が 増える方向へ歩く。増加しなくなるか、10 ステップ進むか、区間境界に達すると停止する。

これは微分を使った精密な最大化や bracket 付き最適化ではなく、固定ステップの 単純な山登りである。getErr は誤差の符号も返すが、search は その符号を零点更新にも収束判定にも使用しない。したがって、隣接する極値の符号が 実際に交互であることは明示的に検証されない。補間零点が区間内に保たれ、誤差が連続で 各零点を横切るなら交互性が期待できる、という前提に依存している。

零点の交換・更新

探索で得た極値の絶対値を

$$E_i = |\varepsilon(m_i)|, \quad i=0, \dots, K \quad (15)$$

とする。コードが収束尺度として用いる spread は

$$S = \frac{\max_i E_i - \min_i E_i}{\min_i E_i} \quad (16)$$

である。 $\min_i E_i = 0$ の場合だけ除算を避ける。

隣接する極値の不均衡から零点移動量を

$$s_i \leftarrow \eta \min\left(\frac{E_i}{E_{i+1}} - 1, 0.25\right) (m_{i+1} - m_i), \quad i=0, \dots, K-1 \quad (17)$$

とし、

$$\xi_i \leftarrow \xi_i - s_i \quad (18)$$

で更新する。ここで緩和係数 η はコード中の変数 delta であり、初期値は 0.25 である。これは minimax 誤差振幅を表す記号ではない。新しい spread が直前より改善しなければ η を半分にする。更新後の零点が $[m_i, m_{i+1}]$ から出そうな場合は、旧零点と対応する極値との 中点へ戻し、点の順序を保つ。

上側の比には 0.25 のクリップがある一方、コードには対称な下側クリップはない。ただし $E_i/E_{i+1} \geq 0$ なので、通常は括弧内の下限が -1 となる。

停止条件

初期値 spread=1e37 から反復し、

$$S \leq 10^{-15} \quad (19)$$

で停止する。この閾値は要求した多倍長精度とは独立に固定されている。 外側反復には最大回数がない。また、緩和係数 η が 10^{-15} 未満になると、精度を増やすよう表示してプロセスを終了する。

返り値の ``error" は、収束時の全点の最大値を再計算したものではなく、 最初の極値位置 m_0 における $|\varepsilon(m_0)|$ である。等振幅状態が正しく得られていれば、これは一様誤差振幅と一致する。

擬似コード

コードの本質を簡略化すると次のようになる。

```
K = n + d + 1
initialize K error zeros xi[0:K-1]
initialize K+1 extrema guesses m[0:K]
initialize search steps; eta = 0.25
spread = large

while spread > 1e-15
    solve R(xi[i]) = f(xi[i]) for coefficients of P and monic Q

    for each interval separated by xi
        walk from m[i] to find a local maximum of abs(R/f - 1)
        store its location m[i] and magnitude E[i]
    end

    new_spread = (maximum(E) - minimum(E)) / minimum(E)
    halve eta if new_spread did not improve
    move each xi[i] according to E[i] / E[i+1]
    keep xi[i] between m[i] and m[i+1]
end
```

連立一次方程式の解法

`simq` は式 (10) の密行列を、行スケーリングを考慮した 部分 pivot 選択付き Gaussian 消去で解く。行 i の最大絶対値を r_i とし、pivot 候補では概ね

$$\frac{|A_{ik}|}{r_i} \quad (20)$$

が最大の行を選ぶ。行自体を物理的に交換せず、`IPS` という行番号配列で 置換を表現し、その後に前進・後退代入する。

一方、行列は $1, x, x^2, \dots$ という単項式基底で作られる。区間の幅、次数、スペクトル比 b/a が大きいと強く悪条件化しやすい。多倍長演算は丸め誤差を軽減するが、基底そのものの条件数は改善しない。Julia 版で高次数を重視するなら、区間を $[-1,1]$ に写して Chebyshev 基底を使う、あるいは列スケーリングを導入する余地がある。

根と部分分数展開

根の計算

収束後、`root` は分子と分母を別々に処理する。多項式 $H(x)$ の根を Newton 法

$$x_{k+1} = x_k - \frac{H(x_k)}{H'(x_k)} \quad (21)$$

で一つ求めるごとに一次因子を除き、残った多項式へ同じ処理を繰り返す。多項式と導関数は Horner 法で評価される。

実装上の重要な制約は次のとおりである。

- 実数演算だけであり、複素根は扱えない。
- Newton 法へ渡す $[-10^5, 1]$ は実際には bracket として強制されず、初期値 $(-10^5+1)/2$ を与えるためだけに使われる。
- 反復上限は各根 10000 回、停止条件は $|\Delta x| < 10^{-20}$ に固定される。
- 根が 0 と返ると失敗扱いなので、真の零根も表現できない。
- 逐次デフレーションは、求めた根の誤差を後続の根へ蓄積しうる。

根を r_j 、分母の根を ρ_j とすると、分母が monic なので

$$R_{n,d}(x) = p_n \frac{\prod_{j=1}^n (x - r_j)}{\prod_{j=1}^d (x - \rho_j)} \quad (22)$$

コード中の `norm` は p_n である。

部分分数展開

`getPFE` と `getIPFE` は $n=d$ の場合だけを扱う。README および出力が採用する規約は

$$R_{n,n}(x) = c + \sum_{i=1}^n \frac{A_i}{x + \beta_i}, \quad c = p_n, \quad (23)$$

であり、実際の分母根 ρ_i に対して

$$\beta_i = -\rho_i, \quad A_i = c \frac{\prod_{j=1}^n (\rho_i - r_j)}{\prod_{j \neq i} (\rho_i - \rho_j)} \quad (24)$$

となる。実装は係数配列として積を展開し、分子と分母の最高次項を相殺して proper fraction にした後、各 pole で評価して留数を得る。最後に shift β_i が小さい順へ並べ替える。

逆数については根と pole を交換して

$$\frac{1}{R_{n,n}(x)} = \frac{1}{c} + \sum_{i=1}^n \frac{\bar{A}_i}{x + \beta_i} \quad (25)$$

を同じルーチンで作る。この形は、格子 QCD で shift の異なる線形系を multi-shift solver に渡す用途を意識したものと考えられる。

コマンドラインプログラムの入出力

実行形式は README では

```
./test y z n d lambda_low lambda_high precision
```

である。すなわち

$$f(x) = x^{y/z}, \quad x \in [\lambda_{\text{low}}, \lambda_{\text{high}}], \quad (26)$$

を次数 (n,d)、指定十進桁数相当の演算で近似する。README の例は

```
./test 1 2 5 5 0.0004 64 40
```

であり、 $\sqrt{x}$ の (5,5) 有理近似を作る。

`approx.dat` にはまず $x^{y/z}$ 、次に $x^{y/z}$ の部分分数係数 c, A_i, β_i を倍精度形式で書く。多倍長で内部計算していても、公開 API と このファイル出力時には `double` へ変換されるため、出力精度はほぼ倍精度までである。

`error.dat` には

$$x, \quad \frac{R(x) - f(x)}{f(x)} \quad (27)$$

を、 $x \leftarrow 1.01x$ という対数的な間隔で記録する。ループ条件が $x < b$ なので上端 b 自体は出力されない。

`main.C` は `force_...dat` と `energy_...dat` という名前も作るが、その後は使用しない。

多倍長演算の実装

`bigfloat` は GMP の `mpf_t` を C++ 演算子で包んだ薄いクラスである。加減乗除・平方根・整数乗は GMP を使い、一般のべきと指数関数には MPFR の関数を使う。ヘッダは古い `mpf2mpfr.h` 互換層に依存しており、現在の GMP/MPFR 環境へそのまま持ち込めるとは限らない。

指定された十進桁数 D は

B=[3.321928094D]

(28)

bit に変換して GMP の default precision に設定される。guard bits は加えない。また、区間端はコンストラクタへ `double` で渡され、 π 、三角関数、初期点変換にも倍精度定数・関数が混在する。したがって「40 桁を指定した」場合でも、すべての入力と前処理が 40 桁で行われるわけではない。

Julia 移植前に把握すべき制約と不具合

以下は単なる文体の古さではなく、再実装時に挙動を保存するか改善するかを明示的に決める必要がある点である。

分類	コード上の性質・問題
収束判定	spread の閾値は常に 10^{-15} 。要求精度に連動せず、外側反復の上限もない。
極値探索	最大 10 回の固定ステップ山登りであり、極値を高精度に bracket・精密化しない。
交互性	誤差の符号を計算するが使用せず、隣接極値が逆符号か検査しない。
線形代数	単項式基底の独自 Gaussian 消去であり、高次数や広い区間で悪条件化しやすい。
根探索	実根のみ。bracket なしの Newton 法と逐次デフレーションで、複素根・重根に弱い。
部分分数	$n=d$ のみ。重 pole や複素共役 pole を扱う設計ではない。
精度境界	入力・公開評価 API・部分分数出力が <code>double</code> で、多倍長結果を倍精度へ丸める。
入力検査	引数個数、 $q \neq 0$ 、 $0 < a < b$ 、非負次数などを検査しない。
エラー処理	ライブラリ内部から <code>exit(0)</code> する箇所があり、失敗でも成功終了 status になる。
戻り値	<code>getPFE / getIPFE</code> は失敗時にも成功時にも 0 を返し、状態を区別できない。
状態管理	再生成前に <code>foundRoots</code> を false に戻さないため、2 回目の根探索失敗後に古い成功状態が残る。
メモリ	<code>main.C</code> は <code>new[]</code> した配列へ <code>delete</code> を使い、未定義動作になる。根探索の早期 return にも一時配列の leak がある。
配列確保	<code>IPS</code> を <code>new int[neq*sizeof(int)]</code> としており、必要量より <code>sizeof(int)</code> 倍多く確保する。
次数不一致	近似生成自体は $n \neq d$ に対応するが、main は部分分数失敗を確認せず配列を出力する。特に $n > d$ では範囲外アクセスの危険がある。
指数因子	式 (3) の k_j は符号付き整数だが、整数べき関数の実体は unsigned exponent 用であり、負の k_j は安全でない。
古い依存関係	Makefile は <code>\$HOME/gmp</code> と <code>mpf2mpfr.h</code> を前提にし、現代的な標準配置や package 管理に対応しない。

さらに、main の $x \leftarrow 1.01x$ という出力ループは $x=0$ なら進まない。分数べきと相対誤差の定義も考えると、現在の用途では区間下端を正に制限するのが自然である。

Julia 版へ移す際の推奨分割

元コードとの数値比較を可能にするには、最初からアルゴリズムを全面的に変更するより、まず「互換経路」と「改善経路」を分けるのが安全である。

1. **対象関数と誤差**: f 、相対誤差、区間を独立した値・関数として定義する。
2. **係数生成**: 零点から式 [\(10\)](#) を組み立てる純粋関数にする。
3. **交換反復**: 極値位置、零点、spread、反復履歴を結果として保存し、最大反復数と要求精度に応じた停止条件を持たせる。
4. **後処理**: 根探索と部分分数展開を係数生成から分離する。近似係数が必要なだけなら、根探索失敗で全計算を失敗させない。
5. **精度**: `setprecision(BigFloat, bits) do ... end` の局所 scope を使い、区間端も文字列から `BigFloat` として parse して精度を保つ。
6. **結果型**: 係数、誤差、収束判定、極値列、根、留数を `named result` に格納し、`Float64` 変換は利用者が明示した場合だけ行う。

互換性確認では、少なくとも次を自動テストすべきである。

- 各補間点で $|R(\xi_i) - f(\xi_i)|$ が作業精度に応じて十分小さいこと。
- 区間内に pole がないこと。
- $K+1$ 個の極値で符号が交互になり、絶対値の spread が許容値以下であること。
- 係数形式、因数形式、部分分数形式が同じ値を返すこと。
- $R(x)$ と逆数近似 $1/R(x)$ が相互に整合すること。
- README の $(p,q,n,d,a,b)=(1,2,5,5,0.0004,64)$ を回帰例にすること。

改善版では、極値探索を bracket 付き次元最適化に替え、誤差符号の交互性を収束条件へ含めることが最優先である。その次に、区間の正規化と安定な多項式基底、複素根を扱える固有値ベースの根計算、重 pole を含む部分分数処理を検討できる。

コード位置との対応

主要な説明と元コードの対応は次のとおりである（行番号は調査時点）。

処理	ファイル	行
反復全体	src/alg_remez.C	110–179
初期点	src/alg_remez.C	264–299
極値探索と零点更新	src/alg_remez.C	301–383
補間方程式	src/alg_remez.C	385–423
Horner 評価と相対誤差	src/alg_remez.C	425–473
Gaussian 消去	src/alg_remez.C	475–588
根探索	src/alg_remez.C	590–682
部分分数展開	src/alg_remez.C	684–752
コマンドラインと出力	src/main.C	13–101
多倍長 wrapper	src/bigfloat.h	16–209

要約

この実装の中心は、 $K=n+d+1$ 個の誤差零点で $P/Q=f$ を解く線形補間と、零点で区切られた $K+1$ 区間の相対誤差極値を等振幅にする零点移動である。収束後の根・部分分数展開まで含め、格子 QCD で利用しやすい $c+\sum_i A_i/(x+\beta_i)$ を生成する。

Julia 化では、まずこの零点交換法を再現して既存結果との回帰比較を可能にし、その上で極値探索、収束検証、線形代数、根探索を段階的に置き換えるのがよい。特に「relative minimax」「分母 monic」「零点補間」「 $n=d$ の部分分数」という四つが元実装の数値仕様の核である。

原資料: [AlgRemez/AlgRemez_analysis.tex](#)。LuaLaTeX がない環境のため、同じ本文をローカル HTML 組版して PDF 化した。